Ejercicio MySQL: ShopDex, consultas sobre una tienda online

En este ejercicio vas a trabajar con la base de datos de **ShopDex**, una tienda online que vende informática, móviles, audio, videojuegos, hogar, deporte y libros. La base de datos ya está creada y llena de datos: tu trabajo es escribir las consultas SQL que alimentan las pestañas de la web de la tienda.
Igual que en GameDex, cada consulta que escribas se convierte en una pestaña con tarjetas. La diferencia es que aquí los datos están repartidos en siete tablas relacionadas y muchas preguntas exigen combinar varias de ellas, agrupar, calcular totales o usar subconsultas.

Objetivos

- Combinar tres y cuatro tablas con JOIN y renombrar columnas con alias.
- Encontrar filas que no tienen pareja en otra tabla con LEFT JOIN … IS NULL o NOT IN.
- Agrupar con GROUP BY, calcular con SUM, COUNT, AVG y ROUND, y filtrar grupos con HAVING.
- Usar subconsultas, incluida una subconsulta correlacionada en el reto.
- Entender la diferencia entre el precio actual de un producto y el precio al que se vendió.

Material

Tu profesor te pasará la carpeta **kit-visor-shopdex**. Dentro encontrarás:

Fichero	Para qué sirve
shopdex.sql	Crea la base de datos shopdex con todas sus tablas y datos. Puedes ejecutarlo tantas veces como quieras para empezar de cero.
.env	Tu usuario y contraseña de MySQL, y tu nombre. Lo tienes que editar.
consultas.js	Las consultas de cada pestaña. Es el fichero que tienes que rellenar y entregar.
server.js, public/	El visor web. No hace falta tocarlo.
README.md	Las mismas instrucciones de puesta en marcha y una tabla con los errores más habituales.

Puesta en marcha

1. Abre shopdex.sql en Workbench y ejecútalo entero (icono del rayo). Comprueba que

tienes 8 categorías, 14 marcas, 40 productos, 20 clientes, 48 pedidos, 100 líneas de pedido y 56 valoraciones.
2. Comprueba que tienes Node.js instalado: en una terminal, node -v debe mostrar la versión 20 o superior. Si no, instala la versión LTS desde nodejs.org.
3. Abre la carpeta del kit en VS Code, abre una terminal (Terminal, New Terminal) y ejecuta npm install. Solo hace falta la primera vez.
4. Edita .env y pon tu MYSQL_USUARIO, tu MYSQL_CONTRASENA y tu NOMBRE.
5. Arranca el visor con npm start. La terminal debe decir: «Conectado a MySQL: hay 40 productos y 48 pedidos».
6. Abre **http://localhost:3000** en el navegador. Para pararlo, pulsa Ctrl + C en la terminal.

El modelo de datos

Tabla	Columnas	Claves ajenas
categorias	id, nombre, icono (un emoji)	Ninguna
marcas	id, nombre, pais	Ninguna
productos	id, nombre, precio, stock, categoria_id, marca_id, fecha_alta	categoria_id → categorias.id marca_id → marcas.id
clientes	id, nombre, apellidos, email, ciudad, fecha_registro	Ninguna
pedidos	id, cliente_id, fecha, estado	cliente_id → clientes.id
lineas_pedido	pedido_id, producto_id, cantidad, precio_unitario	pedido_id → pedidos.id producto_id → productos.id
valoraciones	id, producto_id, cliente_id, puntuacion, comentario, fecha	producto_id → productos.id cliente_id → clientes.id

Antes de empezar, dibuja en papel cómo se relacionan las tablas. Fíjate en estos detalles:

- **Un pedido tiene varios productos.** La relación entre pedidos y productos es de muchos a muchos y se resuelve con la tabla intermedia lineas_pedido, cuya clave primaria es la pareja (pedido_id, producto_id).
- **Lo que cuesta una línea es cantidad * precio_unitario.** precio_unitario es el precio al que se vendió, que puede ser más bajo que el precio actual del producto porque hubo rebajas. Para calcular lo facturado usa siempre precio_unitario.
- **El estado de un pedido** puede ser pendiente, enviado, entregado o cancelado. Los pedidos cancelados no cuentan como ventas.
- **Una valoración** es la puntuación de 1 a 5 que un cliente da a un producto. Un cliente solo puede valorar una vez cada producto.
- Hay datos pensados para los LEFT JOIN: una categoría y una marca sin productos, productos que nunca se han vendido y clientes que nunca han comprado.

Cómo comprueba la web tus consultas

Escribe cada consulta entre las dos comillas invertidas del campo sql de su pestaña en consultas.js, guarda y recarga la web con F5. No hace falta reiniciar el visor. Te recomendamos probar cada consulta antes en Workbench.
Debajo de la descripción de cada pestaña aparece una lista de comprobaciones, en verde si se cumplen y en rojo si no:

- el número de tarjetas (filas) que debe salir;
- las columnas que debe tener el resultado, con ese nombre exacto (usa alias con AS);
- el orden en que deben salir las filas;
- en algunas pestañas, cuánto debe sumar una columna. Si el número de tarjetas cuadra pero la suma no, el error está en el cálculo.

El punto junto al nombre de la pestaña se pone verde cuando todas las comprobaciones se cumplen. Arriba, «Tu progreso» indica cuántas pestañas tienes bien. **No modifiques** los campos tarjetasEsperadas, columnas, orden ni suma: son las comprobaciones.
La web elige el tipo de tarjeta según las columnas: con nombre y precio dibuja un producto, con estado un pedido y con apellidos un cliente. El visor solo ejecuta consultas SELECT, así que no puedes estropear los datos desde ahí.

Las consultas

1. Catálogo (40 tarjetas)

Todos los productos con el nombre de su categoría (alias categoria), el icono de la categoría y el nombre de su marca (alias marca), del más caro al más barato.
Practicas: JOIN de tres tablas y alias de columna. Las tres tablas tienen una columna nombre: pon delante el alias de la tabla.

2. Últimas unidades (10 tarjetas)

Los productos de los que quedan menos de 5 unidades, también los agotados, del que menos stock tiene al que más.
Practicas: WHERE y ORDER BY.

3. Hecho en España (6 tarjetas)

Los productos de marcas españolas que cuestan entre 20 y 100 €, ambos incluidos, con el nombre de la marca (alias marca), del más barato al más caro.
Practicas: JOIN y BETWEEN.

4. Pedidos en curso (7 tarjetas)

Los pedidos que están pendientes o enviados, con el nombre del cliente (alias cliente) y su ciudad, del más antiguo al más reciente.
Practicas: IN, JOIN y ordenar por fecha.

5. Sin estrenar (7 tarjetas)

Los productos que no se han vendido nunca, es decir, que no aparecen en ninguna línea de pedido.
Practicas: LEFT JOIN … WHERE … IS NULL, o bien NOT IN con una subconsulta. Intenta resolverla de las dos formas.

6. Por encima de la media (14 tarjetas)

Los productos que cuestan más que el precio medio de todos los productos, del más caro al más barato. El precio medio lo tiene que calcular la consulta: no vale escribir el número a mano.
Practicas: subconsulta que devuelve un único valor.

7. Ventas por categoría (7 tarjetas)

Cada categoría que tenga ventas, con su nombre e icono, las unidades vendidas (alias unidades) y lo facturado (alias facturado, calculado como cantidad × precio_unitario). No cuentes los pedidos cancelados. De más a menos facturado.
Practicas: JOIN de cuatro tablas, GROUP BY y SUM de una expresión. La web comprueba la suma de unidades y de facturado.

8. Los más valorados (4 tarjetas)

Los productos que tienen 3 valoraciones o más y una puntuación media de 4 o más. Muestra el nombre, el precio, la media redondeada a un decimal (alias media) y el número de valoraciones (alias valoraciones). De mayor a menor media.
Practicas: AVG, ROUND, COUNT y HAVING.

9. Top 5 clientes (5 tarjetas)

Los 5 clientes que más han gastado, sin contar pedidos cancelados. Columnas: nombre, apellidos, ciudad, cuántos pedidos ha hecho (alias pedidos) y cuánto ha gastado (alias gastado). Del que más gasta al que menos.
Practicas: GROUP BY, LIMIT y COUNT(DISTINCT …). Piensa por qué COUNT(*) no da el número de pedidos cuando haces JOIN con lineas_pedido.

Retos opcionales

Reto 1. Ventas por mes (10 tarjetas)

Cada mes con ventas, con el mes en formato 2026-01 (alias mes), cuántos pedidos hubo (alias pedidos) y lo facturado (alias facturado), sin contar los cancelados. En orden cronológico.
Practicas: DATE_FORMAT y agrupar por una expresión.

Reto 2. Caros para su categoría (14 tarjetas)

Los productos que cuestan más que la media de su propia categoría, no la de toda la tienda, con el nombre de la categoría (alias categoria). Ordenados por categoría.
Practicas: subconsulta correlacionada, es decir, una subconsulta que usa una columna de la consulta de fuera.

Entrega

Entrega estos dos ficheros:

- Tu consultas.js con todas las consultas que hayas resuelto.
- Una captura de pantalla de la web en la que se vea tu nombre en el título y el recuadro «Tu progreso».

Evaluación

Parte	Puntos
Consultas 1 a 4	0,75 cada una (3 puntos)
Consultas 5 y 6	1 cada una (2 puntos)
Consultas 7, 8 y 9	1,5 cada una (4,5 puntos)
Limpieza del SQL: alias de tabla, sangrado, nada escrito a mano que deba calcular la consulta	0,5 puntos
Retos opcionales	Hasta 1 punto extra (0,5 cada uno)

Una consulta cuenta como correcta cuando todas sus comprobaciones salen en verde y está resuelta con SQL, sin valores copiados de los resultados.

Si algo falla

Mensaje	Qué hacer
MySQL no acepta tu usuario o tu contraseña	Revisa .env, guarda y reinicia el visor (Ctrl + C y npm start).
No consigo conectar con MySQL	MySQL no está arrancado. Ábrelo desde Workbench.
No existe la base de datos «shopdex»	Ejecuta shopdex.sql en Workbench.
Hay una columna que está en dos tablas	Pon el alias de la tabla delante: p.nombre, c.nombre.
Con GROUP BY, todas las columnas…	Añade al GROUP BY las columnas del

Mensaje	Qué hacer
	SELECT que no estén dentro de COUNT, SUM o AVG.
No puedes usar COUNT, SUM, AVG… dentro del WHERE	Pasa esa condición a un HAVING después del GROUP BY.
No puedo leer consultas.js	Falta una comilla invertida o una coma en consultas.js.
El puerto 3000 ya está en uso	Ya tienes el visor abierto en otra terminal. Ciérralo o cambia PUERTO_WEB en .env.